

CMPT 276

Phase 2 Report

Authors:

M. Vivan Prasad

Yoonsang You

Anthony Yuen

Artyom Borodovitsyn

Describe your overall approach to implementing the game

The implementation phase of *Vault Escape* involved translating our initial design into functional Java code. We followed our initial UML diagrams and started implementing our classes. This phase required us to implement core game logic, including player movement, reward collection, enemy interactions, and game state management. The first steps to implementation involved making player movement, collision, enemies and map generation. We created frameworks to aid in constructing and displaying the base game (i.e. sprite, entity and panel classes). Ultimately, the focus of our approach is to create a modular and maintainable codebase, ensuring that each class has a single responsibility in accordance with object-oriented design principles.

State and justify the adjustments and modifications to the initial design of the project (shown in class diagrams and use cases from Phase 1).

As we transitioned from design to implementation, several adjustments were made to our initial design based on practical considerations and feedback from team members:

1. Class Structure Adjustment

- Specifically, we incorporated cardinality and visibility into our UML diagrams to enhance clarity and understanding of the relationships between classes.

2. Gameplay Mechanics Enhancements:

- Enhanced the enemy AI behavior for dogs, allowing them to chase the player when within a specific proximity, adding a layer of strategy and tension to gameplay.
- Modified scoring penalties to be more significant upon encountering enemies, emphasizing the importance of stealth in the game.

3. Sprite Classes and Generator Classes

- The sprite class encompasses any drawn image that contains an x,y, width, height and image. Since almost every class from enemies to rewards to doors would require this base class to draw onto the screen.
- Since we did not account for camera movement in our game, Sprite2D acted as a way to calculate and draw tiles relative to the player's position in the global world position.
- The generator classes handle the functionality and use cases of the rewards and enemies. These serve to not keep the rewards and enemies abstract while implementing the functionality as a factory process. Moreover, we could control how the spawning of these rewards and enemies would occur from the GamePanel, as opposed to each enemy/reward handling itself upon spawning in.

4. Panel Classes

- The panel classes act as the different menus that can be interacted with. With the use of external libraries, these panel classes handle overlay and main UI elements. The GamePanel is where the main game occurs.

Explain the management process of this phase and the division of roles and responsibilities.

We adopted the **scrum** methodology for our management process, emphasizing collaborative teamwork and iterative progress. Key elements of our approach included:

- **Equal Work Distribution:** All team members participated equally in coding, testing, and reviewing. This collaborative effort helped ensure that everyone had a thorough understanding of the entire codebase.
- **Daily Check-Ins:** We held meetings on Discord to discuss progress, share any obstacles encountered, and plan the day's tasks ahead.
- **Sprints:** We organized our work based on the course deadlines and overall school schedule, each with specific goals and deliverables. At the end of each sprint, we reviewed our progress and adjusted our plans as needed.

This management process facilitated effective communication, encouraged collective problem-solving, and enabled us to respond quickly to challenges.

List external libraries you used, for instance for the GUI and briefly justify the reason(s) for choosing the libraries.

We incorporated several external libraries to facilitate our development process:

- **Java Swing:** Used for building desktop applications. It provides a rich set of components (such as buttons, labels, panels, etc.) that are platform-independent and allow for a flexible and customizable UI design. It is integrated with Java, perfect for Java applications like our application.
- **Java AWT (Abstract Window Toolkit):** Often used alongside Java Swing, providing a basic framework for building GUI applications in Java. This handles event handling and drawing capabilities. In our project, it was used for rendering graphics and handling key events.
- **Java ImageIO:** Java standard library that is used for reading and writing images. It allowed us to load background images for our panels and overlays, enhancing the visual appeal of our game.
- **Java IO (Input/Output):** Reading resources, such as fonts and images, from the file system. It is an essential part of our application to dynamically load resources needed for our application at runtime.
- **JUnit:** For unit testing our code (for Phase 3). This library was chosen due to its widespread use and compatibility with Java, allowing us to

Describe the measures you took to enhance the quality of your code

To ensure high-quality code throughout the implementation, we adopted several practices:

- Conducted regular code reviews to identify potential issues early and maintain consistent coding standards.
- Utilized JUnit for systematic testing of game functionalities, ensuring each component worked as intended before integration.
- Employed JavaDoc comments and descriptive inline comments throughout the codebase to improve readability and maintainability.

We adhered to object-oriented programming principles where applicable. However, not every solution requires OOP design principles, such as in the Panel classes where they do not necessarily apply. Panel classes use a lot of parameter passing to interact with things, and because of their concrete implementation, are more structural than object-oriented.

Over the many commits of this phase, it was imperative to ensure game systems were concise and intuitive so code could be improved and reimplemented. So, many code structures were reworked and improved to ensure clarity.

Any duplicate fragments of code in entities, rewards, and tiles, were simplified by abstraction and encapsulation into the base Sprite class, which they all now inherit from in one way or another. Furthermore, the base Entity class allowed for polymorphism for non-moving entities from rewards to complex dog enemies, and even the framework for the exit door!

Discuss the biggest challenges you faced during this phase

During this phase, several significant challenges emerged that impacted the overall development process of the game.

The biggest challenge during this phase was the enemy behavior and GUI. The initial design of the camera was good in concept, but in execution, issues arose upon interacting with the camera. Upon detection, our initial plan was to leak the player's location to the dog entity allowing it to chase it globally around the map. During implementation of such logic, we realized that it required an understanding of nodes and searching algorithms such as Depth First Search and Breadth For Search. As a result, we decided to apply another form of penalty which was a speed multiplier for enemy entities, thus making the game much more difficult to complete.

Like enemy design, finding a balance for the game difficulty was challenging, particularly regarding the collection of rewards within the time limit. While testing the game, we found it too difficult to collect rewards due to the tight time limit imposed by the gameplay. This required us to do several iterations of gameplay to adjust the difficulty level. This will be an ongoing challenge when further implementing different and new features to our game.

Our project required various classes to be integrated, and maintaining independence among classes was another hurdle during design and implementation of our game. Our group made efforts to reduce coupling and enhance cohesiveness among classes, leading to complications in how components interact with one another. This required us to do more code review and further discuss the design of our project to ensure classes were cohesive and independent from each other.

Because of the implementation of these various classes, small debugging issues arose during the integration of various panels and event handlers. Ensuring that all components displayed as intended involved meticulous troubleshooting and testing. This process was time-consuming to debug and trace the program. Minor code can disrupt the flow of the game. For example, one of our overlays was displayed during our menu which was not intended.

These minor debugging issues can heavily impact the user experience playing our game. Likewise, finding a design to look appealing to our users was a challenge. We kept the external library used to display the user interface very minimal to avoid any complications and allow more complex features to be further implemented to improve the game. Although we tried to keep the user interface minimal, we found implementing UI, such as button logic, to be a slight challenge.

Additionally, working with sprites for various entities presented as a challenge for us. Our team faced difficulties in creating and animating sprites initially, as this was an unfamiliar area for our team. Thus, we were required to do additional research on how to create and utilize sprites, and handle animations and interactions between our entities. Example of this challenge was the sprites of the Dog entities twitching frantically left and right when the player was in range of Dog's vision. This challenge highlighted the importance of visual elements in enhancing the game's overall appeal.

Another challenge that would impact our user's experience was the implementation of background music (BGM) and sound effects (SFX). Java's file size limits and format compatibility issues created hurdles in managing audio resources effectively. Ensuring that the audio files were in a suitable format and within the allowed size constraints required extensive testing and adjustments. This required us to find solutions such as compressing the files into wav for integrating sound into the game experience.

With these many challenges, time management also proved to be a significant challenge throughout the development process. Balancing project work with academic responsibilities necessitated careful scheduling and prioritization. Finding consistent blocks of time to focus on implementation was often difficult, leading to delays in progress.

Overall, these challenges underscored the complexities involved in game development, emphasizing the need for effective planning, collaboration, and problem-solving throughout the project lifecycle.